

Search-and-Rescue Marker Finder – Design

ROS 2 Humble · Gazebo Classic 11 · Nav2 · OpenCV 4.5.4 (cv2.aruco) · Docker | Repository:
github.com/sushishariq/search_rescue_robot

A TurtleBot3 Waffle patrols a 12×10 m simulated building with four rooms. It localizes with AMCL on a map built once with slam_toolbox, follows a Nav2 waypoint route, detects ArUco victim markers with classical OpenCV, projects each detection into the map frame through TF, and keeps a deduplicated victim registry that is exposed through the /get_victims service and exported to JSON/CSV. The whole stack starts with docker compose up sar, which runs a single mission launch file.

1. Architecture

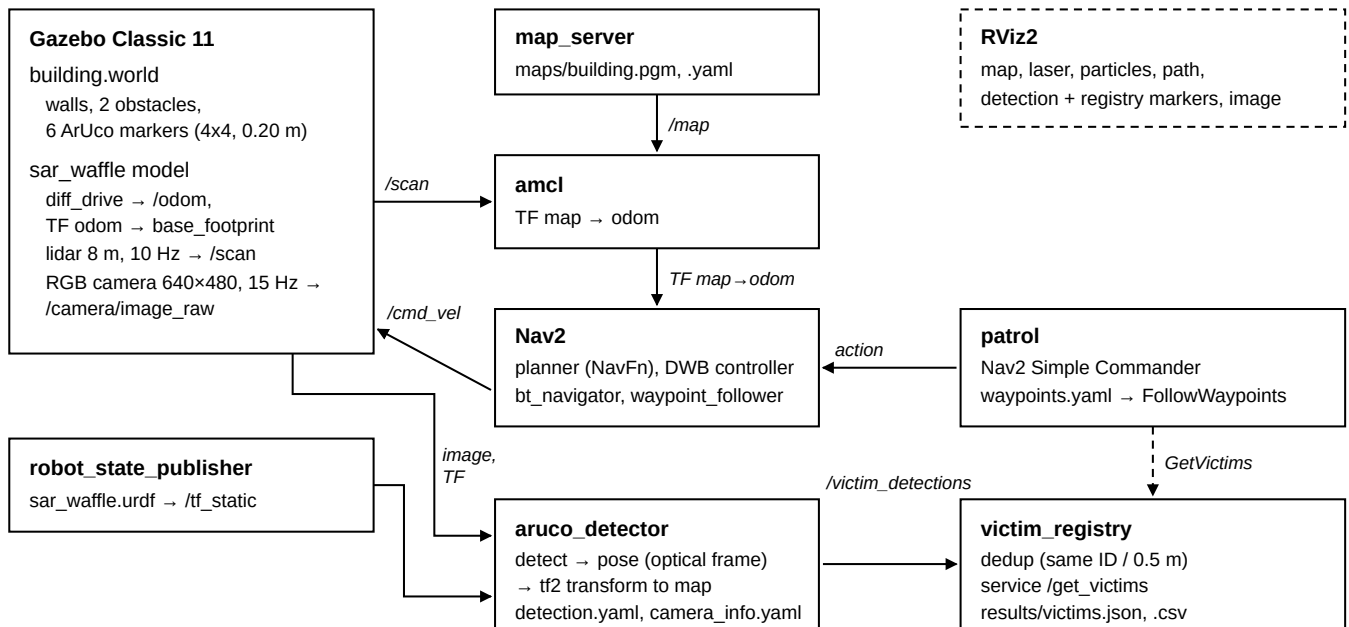


Figure 1. Nodes and main interfaces. All nodes use simulated time. The mission launch file (sar_bringup/launch/mission.launch.py) starts every box shown.

Package	Contents
sar_bringup	world and marker generators, robot model (SDF + URDF), saved map, AMCL/Nav2/SLAM parameters, camera calibration, launch files (sim, slam, nav, mission)
sar_perception	aruco_detector: detection, pose estimation, camera → map projection, RViz markers, debug image
sar_mission	victim_registry (dedup, service, JSON/CSV export) and patrol (waypoints); dedup rules unit-tested
sar_interfaces	msg/Victim (id, map position, range, observations), srv/GetVictims

2. Environment and sensor wiring

The world and markers are generated by scripts, so marker size and positions are defined once and the ground truth is known. Each marker is a thin static box textured with a DICT_4X4_50 image: a 0.20 m black square inside a 0.05 m white quiet zone. The robot is a copy of the TurtleBot3 Waffle with three changes, each found by measuring:

- Optical frame:** the stock camera plugin stamps images with camera_rgb_frame (x forward), while OpenCV poses are in the optical frame (z forward). frame_name is set to camera_rgb_optical_frame so the TF chain is correct without a rotation patch in code.
- Camera pose:** the SDF camera sat 1 cm below the URDF camera. The SDF pose was set to match, and the URDF rotation uses exact $\pi/2$ values.
- Resolution and lidar:** the camera runs at 640×480 and 15 Hz instead of 1920×1080 at 30 Hz. camera_info.yaml follows from the field of view: $f = 640 / (2 \cdot \tan(1.02974/2)) = 565.6$ px, $c = (320.5, 240.5)$, zero distortion. The lidar range was raised to 8 m at 10 Hz so SLAM and AMCL see through doorways.

3. Camera-to-map TF chain

`cv2.aruco.detectMarkers` finds the four corners of each marker, and `estimatePoseSingleMarkers` solves for the marker pose from the known side length (0.20 m) and K. The result, a rotation vector and translation vector, is expressed in `camera_rgb_optical_frame`: x right, y down, z forward (REP-103 camera convention). The detector puts this pose in a `PoseStamped` with the **image's** header (frame and timestamp) and calls `tf2 Buffer.transform(pose, "map")`. `tf2` then composes these transforms:

Transform (parent → child)	Published by	Nature
map → odom	AMCL	dynamic; corrects odometry drift from laser/map matching
odom → base_footprint	Gazebo diff_drive plugin	dynamic, 30 Hz
base_footprint → base_link → camera_link → camera_rgb_frame	robot_state_publisher (URDF)	static; camera at (0.069, −0.047, 0.117) m
camera_rgb_frame → camera_rgb_optical_frame	robot_state_publisher (URDF)	static rotation RPY (−90°, 0, −90°)

$$p_{\text{map}} = T_{\text{map-odom}}(t) \cdot T_{\text{odom-base}}(t) \cdot T_{\text{base-optical}} \cdot p_{\text{optical}} \quad (t = \text{image timestamp})$$

Why the image timestamp matters. The robot keeps moving while an image is processed. Looking up TF at the image time uses the robot pose at the moment the photo was taken; using "latest" would place markers wrongly whenever the robot turns.

Why REP-105 splits the chain. A TF frame can have only one parent, so odometry owns `odom → base` (smooth, drifting) and AMCL owns `map → odom` (the correction). **Implementation detail.** The TF subscriptions and the image callback are in separate callback groups on a multi-threaded executor, so TF keeps arriving while an image callback waits up to 0.2 s for a transform. A first single-threaded version fell behind TF and failed with extrapolation errors. Detections farther than 3.5 m or smaller than 20 px per side are dropped because pose noise grows with range.

4. Deduplication logic

The detector publishes one `Victim` message per marker per frame, so a single marker yields 50–370 detections during a patrol. `registry_core.py` (pure Python, unit-tested) reduces them to one entry per victim. For each detection (id, x, y, range):

1. **Same ID → same victim.** Merge into the nearest victim with that ID. *Guard:* if that victim is already confirmed and the detection is more than **1.0 m** away, reject it as an outlier (for example, a transform taken during a localization jump).
2. **Nearby position → same victim.** Otherwise, if any victim lies within **0.5 m**, merge into it. This absorbs a misread ID. A victim's ID is the majority vote of its observations, so a rare misread cannot rename it.
3. **Otherwise → new victim.**

Position estimate: a weighted running mean with weight $1/\text{range}^2$, so close, more accurate observations dominate (the minimum range used for weighting is 0.5 m). **Confirmation:** a victim is reported by the service and written to file only after **3** observations, which filters one-frame false positives. **Why 0.5 m:** it is several times the observed projection error (≈ 8 cm) yet far below the marker spacing (> 2 m), so two real victims cannot merge. Results are written atomically every 2 s while they change and again on shutdown.

5. Results (real run, victims.json)

ID	map x	map y	obs.	error
0	−3.970	4.847	182	7.9 cm
1	5.872	2.921	156	9.2 cm
2	3.946	−4.868	127	7.5 cm
3	−5.888	−1.981	148	3.7 cm
4	1.145	−1.032	44	7.3 cm
5	−1.132	0.016	372	9.3 cm

6/6 victims found, no duplicates, 0 outliers rejected; mean error **7.5 cm** against ground truth.

6. Navigation and error sources

Map: built with `slam_toolbox` (online async) while Nav2 drove to clicked goals. Because Gazebo's diff drive reports odometry in world coordinates, the saved map frame coincides with the world frame, so results compare directly with ground truth.

AMCL: `alpha1–4 = 0.05` (low odometry noise in simulation), 120 beams, updates every 0.1 m / 0.1 rad, initial pose preset. Measured pose error is 5–10 cm, the main contributor to projection error. Pose estimation alone is within 1 cm at 1.5 m.

Nav2: inflation radius 0.45 m with scaling factor 5 so the 1.2 m doorways stay passable; waypoint poses face each marker from about 1.7 m, with a 2 s pause at each.

Limitations: marker textures ignore lighting; the stock Waffle can yaw slowly for a few seconds after stopping (contact physics; TF follows the true motion).